

Assistant-Based Integrity Auditing Scheme With Privacy Protection Function for Cloud Storage

Kaijing Ling^{ID}, Jie Gao^{ID}, Jinyong Chang^{ID}, and Ru Meng

Abstract—Cloud audit scheme is a mechanism for verifying the integrity of user data stored on cloud servers. Existing audit schemes often require the data owner (DO) to generate tags, which will be used in the later audit phase, on the data files to be stored itself. However, this mode still has the following issues: 1) the calculation and generation process of tags imposes a significant computational burden on DO and 2) frequent use of the private key for tag generation exposes it to increased security risks. To address these issues, this article proposes an assistant-based tag collaborative generation process. Specifically, we introduce an assistant that holds a key derived from the DO's private key and performs the bulk of the tag computation. In this way, the DO only needs to carry out a few lightweight operations to complete the final authentication tag. Moreover, the DO can revoke or update the assistant at any time without changing the original public/secret key pair. In addition, in order to achieve privacy protection against the auditor during the audit process, this article also proposes a transparent audit based on Blockchain. Finally, the security analysis indicates that our scheme achieves the required security in terms of cloud audit reliability, tag collaborative computing security, and privacy protection. The performance analysis also shows that it has certain advantages compared to similar schemes, especially in saving DO's computation costs.

Index Terms—Assistant, blockchain, cloud storage, integrity auditing, privacy protection.

NOMENCLATURE

Symbols	Definitions
q	Prime number.
Z_q	Finite field $[0, 1, \dots, q-1]$.
G_1, G_2	Two cyclic groups with the prime order q .
e	Bilinear map from $G_1 \times G_1$ to G_2 .
msk	Master secret key.
psk_1, psk_2	Two partial secret keys.
M	Data file.

Fname	Name of Data file.
n	Number of blocks in M .
s	Number of sectors in data file block.
M_i	Data file block.
N_j	Challenge data file block.
$\bar{\sigma}_i$	Partial tag of M_i .
$\bar{\Sigma}$	Partial tag set.
σ_i	Tag of M_i .
Σ	Authentication tag set.
ϕ	Pseudorandom permutation.
ψ	Pseudorandom function.
k_1	Key of ϕ .
k_2	Key of ψ .
T_{mul}	Time of multiplication on G_1 .
T_{exp}	Time of exponentiation on G_1 .
T_{pair}	Time of bilinear pairing on G_1 .
T_H	Time of hash computation.
$T_{Z_q^+}$	Time of addition on Z_q .
$T_{Z_q^*}$	Time of multiplication on Z_q .
T_{prp}, T_{prf}	Time of PRP and PRF.
$ G_1 , Z_q , Z_q^* $	Bit-lengths of elements on G_1, Z_q, Z_q^* .

I. INTRODUCTION

WITH the explosive development of mobile Internet, Internet of Things, and artificial intelligence computing, the total amount of global data is also growing rapidly. Traditional local storage faces severe challenges in terms of capacity scalability, maintenance costs, and cross-domain collaboration [1]. Distributed cloud storage technology provides a technical foundation for solving these problems [2]. In the cloud storage mode, data owner (DO) rents a certain amount of storage space on the cloud server provider (CSP) for data storage, and its core significance lies in achieving a paradigm shift from “physical device dependence” to “service-oriented supply” [3]. This mode not only eliminates traditional expenditures, such as hardware procurement and computer-room maintenance, which saves costs for users, but also supports convenient access and multiterminal collaboration anytime, anywhere. Therefore, cloud storage has developed into the infrastructure of the digital economy and has become the core engine driving social digital transformation.

It should be emphasized that, after uploading data to CSP, DO naturally deletes this data to save local storage space, which leads to the lose physical control over these data for DO. Once CSP maliciously deletes or accidentally damages them, it will cause incalculable losses to DO [4]. In fact, in

Received 18 June 2025; revised 14 September 2025 and 12 October 2025; accepted 21 October 2025. Date of publication 30 October 2025; date of current version 23 December 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62476212; in part by the Key Research and Development Program Project of Shaanxi Province under Grant 2025CY-YBXM-007; in part by the Natural Science Foundation Project of Shaanxi Province under Grant 2025JCYBMS-774; and in part by the Open Project of State Key Laboratory of Integrated Service Networks under Grant ISN25-02. (Corresponding author: Ru Meng.)

Kaijing Ling is with the Department of Mathematics, Changzhi University, Changzhi, Shanxi 046000, China.

Jie Gao and Jinyong Chang are with the School of Information and Control Engineering, Xi'an University of Architecture and Technology, Xi'an, Shaanxi 710055, China.

Ru Meng is with the College of Cyber Security, Jinan University, Guangzhou 510632, China (e-mail: mengru93@stu2021.jnu.edu.cn).

Digital Object Identifier 10.1109/IJOT.2025.3627259

2327-4662 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.

See <https://www.ieee.org/publications/rights/index.html> for more information.

recent years, there have been frequent cases of data loss or damage stored on CSPs [5]. Hence, it is necessary for DO to conduct periodic audits of the integrity of the outsourced data. Once partial data loss or damage is discovered, it will initiate a claim or other compensation to CSP. Technically speaking, the most typical way to efficiently audit the integrity of data stored on CSP is to authenticate the data file during the storage phase, and then store the original data files and authentication files together on CSP. During the audit phase, DO itself or its commissioned third-party auditor (TPA) launches challenge queries through sampling. CSP calculates and returns the corresponding proof based on the stored data file and authentication tag after receiving the challenge message. Finally, the auditor verifies the correctness of this proof to determine whether the challenged blocks of the data file are integral. In fact, multiple existing integrity audit schemes use this mechanism for auditing [6].

Here, we remark that the above audit model often has the following problems. First, the authentication tag of the data file is generated by DO, which will bring heavy computational costs to DO. Second, frequent use of DO's private key for tag generation poses a risk to the security of this private key [7]. Third, the audit process may leak the content of user-stored data when the challenge message is not sufficiently random.

For the first problem, the existing method is to add a proxy server between DO and CSP, which is responsible for generating an authentication tag for DO's data file [8]. Naturally, the authentication tag depends on the private key of the proxy server instead of the private key of the DO. Obviously, this model has a serious vulnerability: the proxy server can modify the original data of DO according to its own wishes and then generate a tag for the modified data. That is, the security of auditing cannot truly guarantee the integrity of DO's original data. The natural solution to the second problem is to regenerate the authentication private key. Along with this, the corresponding public key also needs to be regenerated and authenticated by the trusted certificate authority (CA), which obviously brings additional computational burden to DO [9]. As for the third problem, in order to prevent DO stored data from being recovered by TPA during the audit process, CSP can add a certain degree of randomness when returning proof, thus satisfying the requirement that even if the challenge message does not have any randomness, the auditor cannot recover the contents of the corresponding data blocks [10]. In order to ensure that the verification process can correctly pass, the increased randomness of CSP needs to be additionally fed back to the auditor. This not only increases the communication overhead of the audit process but also causes an increase in computational cost of the verification process.

In this article, we propose an improved scheme to address the aforementioned problems.

- 1) To reduce the computational burden on the DO, we introduce an entity called the Assistant, which is responsible for helping the DO generate the authentication tag. Indeed, the Assistant carries out most of the tag-generation computations, while the DO only performs a small number of lightweight operations.

- 2) To address the second problem, we design a two-party signature protocol that enables the DO and the Assistant to jointly compute the tag without ever exposing the DO's real private key. In addition, the Assistant can be revoked or updated whenever the DO wishes.
- 3) Finally, to achieve trustworthy auditing, we leverage blockchain technology, whose transparency, traceability, and immutability make it an ideal audit log. To further increase the unpredictability of the auditor's challenge, we incorporate a pseudorandom function (PRF), a pseudorandom permutation (PRP), and an audit timestamp into the challenge generation process.

Contributions: Based on the above analysis, our contributions can be summarized as follows.

- 1) A mechanism for collaborative generation of an authentication tag will be introduced between DO and the Assistant. Concretely, DO issues a partial private key to this Assistant, which can be used to generate a partial authentication tag for the data file. Then, DO only needs to perform a small amount of calculation to obtain the full authentication tag.
- 2) The efficient revocation and update of the Assistant is given. In particular, DO can easily revoke the permissions of some Assistant according to its own needs, and can also efficiently set up a new Assistant while ensuring that its private key remains unchanged.
- 3) The Blockchain-based privacy protection in the audit process will also be discussed. More specifically, using PRF and PRP together with the audit timestamp to generate a challenge message ensures that the audit process cannot recover the contents of the challenged data blocks.
- 4) Finally, the security and performance analysis on the designed scheme are also given. More precisely, we will discuss the audit reliability, security of the collaborative authentication process, and privacy protection of the auditing process. The performance analysis shows that our scheme has certain advantages in terms of computational, communication, and storage costs, especially for DO.

II. RELATED WORKS

The development of cloud auditing is closely related to the security needs of cloud storage. Early research mainly focused on data storage security and auditing mechanisms in cloud computing environments. In 2007, Ateniese et al. [11] proposed a provable data possession (PDP) scheme, which uses the homomorphic property to verify data integrity and reduce the communication overhead. Shacham and Waters [6] gave the proof-of-retrievability schemes with full proofs of security against arbitrary adversaries in the Juels–Kaliski model. By delegating the verification to a TPA, the protocol reduced the verification burden on DO. In a subsequent work, Ateniese et al. [12] designed a partially dynamic PDP (DPDP) protocol, which utilizes random masking to achieve blind auditing and aggregates data tags using homomorphic labels for batch auditing. Wang et al. [13] proposed a PDP

protocol based on bilinear pairings and the discrete logarithm problem, supporting the verification of TPA. The concept of Blockchain, which was tied to the advent of Bitcoin, was first introduced by Nakamoto [14]. Due to the decentralized and trustworthy of Blockchain, more and more cloud auditing systems use Blockchain instead of TPAs for data integrity auditing. Zhang et al. [15] proposed a Blockchain-enabled decentralized consistency verification scheme, which can verify whether the data involved in the calculation is real-time by leveraging a counting bloom filter. Wang et al. [16] proposed a Blockchain-based dynamic cloud data integrity auditing scheme, which reduces the communication overhead during auditing and update phases through extending the rank-based merkle hash tree (RMHT) for dynamic update of stored data and using a nonleaf node sampling strategy. Din et al. [17] presented an approach to enhancing data security in IoT through the integration of homomorphic encryption and blockchain technology. The existing cloud audit schemes require DO to spend a lot of computational cost in the tag generation phase. The proposed scheme can effectively reduce the DO's computational cost by introducing the Assistant.

A. Proxy Tag Generation

In the tag generation phase of cloud auditing, considering the computing burden of users to generate authentication tags, Mambo et al. [8] proposed the concept of proxy signature. In the proxy signature scheme, the user delegates its signing authority to the proxy signer, and the proxy signer can generate valid signatures that can pass the verification. Lin et al. [18] constructed the first linearly homomorphic proxy signature scheme, which allows the proxy signer to generate a linearly homomorphic signature on behalf of the original signer. Wang and Wu [19] improved the security model of the linearly homomorphic proxy signature scheme and presented a Blockchain-based linearly homomorphic proxy signature scheme. Zhang et al. [20] proposed the blockchain-based proxy-oriented data integrity checking mechanism in cloud-assisted intelligent transportation systems. The mechanism enabled the data manager in the traffic information control center to delegate the proxy to produce the signatures of encrypted data and outsource them to the cloud server. It is worth mentioning that in proxy signature schemes, the signature is generated by the proxy signer. Same as proxy tag generation schemes, our proposed scheme introduces a new server to reduce DO's computational overhead, but the assistant-based integrity auditing scheme enables efficient assistant revocation and replacement. Meanwhile, by changing the partial secret key, instead of changing the system parameter. Different from the proxy signature scheme, the proposed scheme reduces the computational overhead of the DO while the final signature is still generated by the DO.

B. Privacy Preserving

The rapid growth of cloud computing has led to significant advancements in data storage, but it has also introduced new challenges in ensuring data privacy. In the process of cloud auditing, to protect data privacy, Wang et al. [21] proposed a

privacy-preserving integrity auditing scheme based on random masking technology, which ensures that the TPA cannot obtain the DO data in the auditing phase. Their scheme utilized homomorphic linear authenticators and random masking to enable TPAs to verify data integrity without learning the data content. Shen et al. [22] presented a shared data integrity auditing scheme with sensitive information hiding by the introduction of a sanitizer to sanitize the data blocks corresponding to the sensitive information. Gao et al. [23] proposed a new paradigm for integrity auditing based on keywords with sensitive information privacy for encrypted cloud data, which is designed for the scenario where the user is concerned about the integrity of a portion of encrypted cloud files that contain keywords. Wang et al. [24] proposed a privacy-preserving time-based auditing scheme for secure cloud storage. In this scheme, TPA can verify the integrity of all the files based on this time period, but it cannot tell how many files and which files the user has generated and uploaded to the cloud. Hu et al. [25] proposed an efficiency identity-based proxy-server-assisted cloud storage audit scheme with sensitive information hiding, where only sensitive locations of user data are blinded without encrypting the entire file. Compared with the aforementioned works, our proposed scheme achieves privacy protection by increasing the randomness of challenge information and introducing blockchain.

C. Multiparty Collaborative Signing

To avoid signature forgery caused by private key leakage, in 2017, Lindell [26] introduced a two-party signing protocol for ECDSA using zero-knowledge proofs. Building on this, He et al. [7] proposed a two-party distributed signing protocol for the IEEE P1363 standard in 2018, allowing two devices to generate a valid signature without reconstructing the entire private key. Mu et al. [27] enhanced the efficiency and security of the signing process with a secure two-party SM9 signing protocol using homomorphic encryption and zero-knowledge proofs. Liu et al. [28] further advanced the field with an asymmetric secure multiparty signing protocol for the IEEE P1363 standard, using Paillier encryption to ensure security against malicious adversaries. In 2024, Liu et al. [29] introduced a secure and efficient two-party collaborative SM9 signature scheme for smart home environments, storing parts of the private key in different devices to reduce leakage risk. Note that in a multiparty signature scheme, the status of the signing parties is equal. In our proposed scheme, DO uses assistants to generate tags to reduce computational overhead. At the same time, an assistant can be revoked and replaced.

III. PRELIMINARIES

In this section, we introduce some basic notions and definitions that will be used in later sections. The symbols/abbreviations and their corresponding definitions used in this article are listed in Nomenclature.

A. Bilinear Mapping

Let G_1, G_2 be two multiplicative cyclic groups with the same prime order q . We call a binary function $e: G_1 \times G_1 \rightarrow G_2$ as a bilinear mapping if the following conditions hold.

- 1) *Nondegeneracy*: For any two generators $g, h \in G_1$, it holds that $e(g, h) \neq 1$.
- 2) *Bilinearity*: Given any two elements $x, y \in \mathbb{Z}_q$, and $g, h \in G_1$, the following equation holds:

$$e(g^x, h^y) = e(g, h)^{xy}.$$

- 3) *Computability*: For any $g_1, g_2 \in G_1$, there exists an efficient algorithm to compute the value $e(g_1, g_2) \in G_2$.

B. Computational Diffie–Hellman Assumption

Let G be a multiplicative cyclic group of prime order q , and g be a generator of G . Given a tuple (g, g^a, g^b) , where $a, b \in \mathbb{Z}_q$. For any probabilistic polynomial time (PPT) adversary $\mathcal{A}$, the advantage of computing g^{ab} for each $\mathcal{A}$ is negligible

$$\Pr \left[\mathcal{A}_{\text{CDH}}(g, g^a, g^b) = g^{ab} : a, b \xleftarrow{R} \mathbb{Z}_q \right] \leq \epsilon.$$

C. Two-Party Collaborative Signature

A two-party collaborative signature includes four algorithms: 1) *Setup*; 2) *KeyDistrib*; 3) *CollabSign*; and 4) *Verify*. The specific descriptions are as follows.

- 1) *Setup*(1^λ) $\rightarrow$ (pk, msk): This algorithm generates the public key pk and the master secret key msk based on the security parameter.
- 2) *KeyDistrib*(msk) $\rightarrow$ ($\text{psk}_1, \text{psk}_2$): This algorithm outputs the corresponding partial private key $\text{psk}_1, \text{psk}_2$ to Alice and Bob.
- 3) *CollabSign*($M, \text{psk}_1, \text{psk}_2$) $\rightarrow$ (σ): In this algorithm, Alice requests signature and interacts with Bob to generate the signature σ for the message M based on each other's partial private key.
- 4) *Verify*(M, σ, pk) $\rightarrow$ 1/0: This algorithm is run to check the validity of the received (M, σ). If it is correct, output 1. Otherwise, output 0.

IV. SYSTEM MODEL, DESIGN GOALS, AND SECURITY MODEL

In this section, we will introduce the system and design goals as well as security models.

A. System Model

1) *System Architecture*: In our proposed system, there are four kinds of entities: 1) DO; 2) Assistant; 3) cloud service provider (CSP); and 4) Blockchain. The conceptual diagram is shown in Fig. 1.

- 1) *DO*: It is the original DO that wants to transmit its data file to CSP for storing. In order to facilitate the integrity audit process in the later stage, it is necessary to generate authentication tags for the data file. Meanwhile, in order to save its computational costs of generating tags, DO assigns this process (of generating tags) to its Assistant.
- 2) *Assistant*: It is designed to assist DO in generating data file tags, thereby saving DO's computational costs. For this, DO needs to grant it authorization. After obtaining authorization, the Assistant can generate partial authentication tags for the DO's data file.

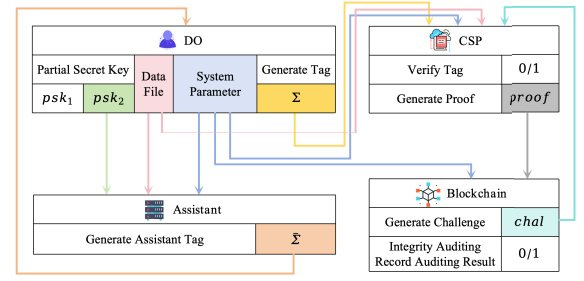


Fig. 1. Conceptual diagram.

- 3) *CSP*: It has huge storage space and thus provides data storage services for different DOs. At the same time, it charges a certain rental service fee. In addition, it also receives auditing of whether the stored data is integral. When the auditor initiates an audit request, it calculates and generates a proof based on the stored data files and authentication tags.
- 4) *Blockchain*: Blockchain can ensure that outsourced data remains intact over time by exploiting the immutability and transparency inherent to distributed ledgers. In order to achieve privacy protection and immutability in the audit process, Blockchain is introduced for our proposed system. Specifically, it initiates a challenge message to CSP, and once the CSP returns proof, it verifies its validity. If the verification passes, then the challenged data blocks are integral. Otherwise, the integrity of these data blocks is compromised. Blockchain immutably records challenge message and corresponding verification results in a decentralized, append-only ledger that follows the DO's audit time period requirements (e.g., daily, weekly, monthly, etc.).

2) *System Algorithms*: In order to maintain the operation of our proposed system and ensure its security, now, we introduce the needed algorithms, which can be divided into the following four phases.

a) *Setup Phase*: In this phase, the system's public parameters, the DO's master secret key, and the Assistant's partial secret key will be generated.

- 1) *Setup*(1^λ) $\rightarrow$ (param, msk): This algorithm is executed by DO to generate the public parameters and its master key. It inputs a security parameter λ , and outputs param as the system parameter and msk as DO's master secret key. For convenience, the system parameter param is implicitly included in other algorithms.
- 2) *KeyGen*(msk) $\rightarrow$ ($\text{psk}_1, \text{psk}_2$): This algorithm is still run by DO. It inputs the master secret key msk , and outputs partial secret keys psk_1 and psk_2 .

b) *Tag Generation Phase*: This phase will generate authentication tags for DO's data file, which includes Assistant's partial generation of tags and DO's final calculation.

- 1) *AssistTag*(M, psk_2) $\rightarrow$ $\tilde{\Sigma}$: This algorithm is executed by Assistant. It inputs the data file M , the partial secret key psk_2 , and outputs $\tilde{\Sigma}$, which consists of auxiliary

information and partial authentication tags of data blocks in M .

- 2) $\text{Tag}(\bar{\Sigma}, \text{psk}_1) \rightarrow \Sigma$: This algorithm is run by DO to generate the final tags for data file M . It inputs $\bar{\Sigma}$ and the partial secret key psk_1 , and outputs Σ .
- 3) $\text{VeriTag}(M, \Sigma) \rightarrow 1/0$: This algorithm is run by CSP to check the validity of the received (M, Σ) . It inputs data file M and corresponding tag set Σ , and outputs 1 if the verification passes and stores it; otherwise, it outputs 0.

c) *Integrity Auditing Phase*: In this phase, Blockchain will periodically audit the integrity of the data stored in the CSP based on DO requirements. This phase includes issuing challenge messages, calculating proofs, and verifying proofs.

- 1) $\text{ChalGen} \rightarrow \text{chal}$: This algorithm is run by Blockchain. It inputs the system parameter, and outputs the challenge message chal .
- 2) $\text{ProofGen}(\text{chal}, M, \Sigma) \rightarrow \text{Proof}$: This algorithm is executed by CSP to generate an integrity proof. It inputs the challenge chal , the data file M and tag set Σ , and outputs the proof proof .
- 3) $\text{Verify}(\text{chal}, \text{proof}) \rightarrow 1/0$: The proof verification algorithm is run by Blockchain. It inputs the challenge chal and the proof proof , and outputs 1 if the proof is valid; otherwise, outputs 0. The result will be recorded on its distributed ledger.

In our system, DO runs Setup algorithm to generate the system parameter and the master secret key. Then, DO executes KeyGen algorithm to generate partial private keys for Assistant. After receiving its partial private key, Assistant executes the AssistTag algorithm to generate partial authentication tags. Subsequently, DO executes Tag algorithm to generate the final tag set corresponding to the data file and sends it to CSP for storage. During integrity auditing, Blockchain executes ChalGen algorithm to generate a challenge message and sends it to CSP. CSP then executes the ProofGen algorithm to generate a proof and returns it to Blockchain, which subsequently executes the Verify algorithm to verify the integrity of the cloud storage.

B. Design Goals

- 1) *Correctness*: The tag set of the data file can be verifiable by CSP if the Assistant correctly generates the partial authentication tags. Meanwhile, CSP can pass the integrity auditing by Blockchain if it correctly saves the data generated and uploaded by DO.
- 2) *Soundness*: Assistant cannot forge a final tag set that can pass the verification. CSP cannot generate a forged proof that can pass the verification by the Blockchain.
- 3) *Efficiency*: In the tag generation phase, Assistant does most of the calculations to generate partial authentication tags. Therefore, DO can cut down the computational cost effectively.
- 4) *Privacy Preserving*: By introducing PRP and PRF in the integrity auditing phase, Blockchain cannot always choose the same data file blocks as the challenged blocks to compute the auditing files.

C. Security Models

We mainly discuss the security model for semi-honest Assistant and CSP. The assistant may attempt to generate a forge tag to pass the CSP's verification and to upload the forage file to CSP. CSP may attempt to forge the proof to pass the integrity verification by the Blockchain.

We define the security games **Game 1** and **Game 2** against semi-honest Assistant and CSP, respectively. These two games are played between the challenger $\mathcal{C}$ and the adversary $\mathcal{A}$. The specific description is as follows.

The security game **Game 1** between the adversary $\mathcal{A}$ and the challenger $\mathcal{C}$ is as follows.

- 1) *Setup*: The challenger $\mathcal{C}$ executes the Setup algorithm to generate the master secret key msk and system parameter param . Then $\mathcal{C}$ sends param to the adversary $\mathcal{A}$ and keeps msk secret.
- 2) *Queries*: $\mathcal{A}$ can interrogate the following queries to $\mathcal{C}$ adaptively.
 - a) *Key-Query*: $\mathcal{A}$ can query the corresponding partial secret key, $\mathcal{C}$ runs the KeyGen algorithm to return the partial secret key psk_2 to $\mathcal{A}$.
 - b) *Hash-Query*: $\mathcal{A}$ can query the hash values of the hash function against the values selected adaptively. $\mathcal{C}$ chooses a random function H and returns hash values to $\mathcal{A}$.
 - c) *Tag-Query*: $\mathcal{A}$ executes AssistTag algorithm and sends assist tag set $\bar{\Sigma}$ to $\mathcal{C}$. $\mathcal{C}$ runs Tag algorithm to return final tag set Σ to $\mathcal{A}$.
- 3) *Forgery*: If $\mathcal{A}$ forges a tag Σ' for a data file M' not be queried, and

$$\text{VeriTag}(M', \Sigma') = 1.$$

Then $\mathcal{A}$ wins this game.

Definition 1: If for any PPT adversary $\mathcal{A}$, the probability of $\mathcal{A}$ winning **Game 1** is negligible, then we call the assistant-based tag generation scheme sound against the semi-honest Assistant.

The security game **Game 2** between the adversary $\mathcal{A}$ and the challenger $\mathcal{C}$ is as follows.

- 1) *Setup*: The challenger $\mathcal{C}$ executes the Setup algorithm to generate the master secret key msk and system parameter param . Then $\mathcal{C}$ sends param to the adversary $\mathcal{A}$ and keeps msk secret.
- 2) *Queries*: $\mathcal{A}$ can perform the following queries to $\mathcal{C}$ repeatedly.
 - a) *Hash-Query*: $\mathcal{A}$ can query the hash values of the hash function against the values selected adaptively. $\mathcal{C}$ chooses a random function H and returns hash values to $\mathcal{A}$.
 - b) *Tag-Query*: For the adaptively selected data blocks, $\mathcal{A}$ can query the corresponding authentication tag. $\mathcal{C}$ returns the tags to $\mathcal{A}$ after running the $\text{Tag Generation Phase}$.
- 3) *Challenge*: $\mathcal{C}$ runs ChalGen algorithm to generate a challenge chal to $\mathcal{A}$ and requires $\mathcal{A}$ to generate a proof as a response. $\mathcal{A}$ executes ProofGen algorithm to return proof to $\mathcal{C}$.

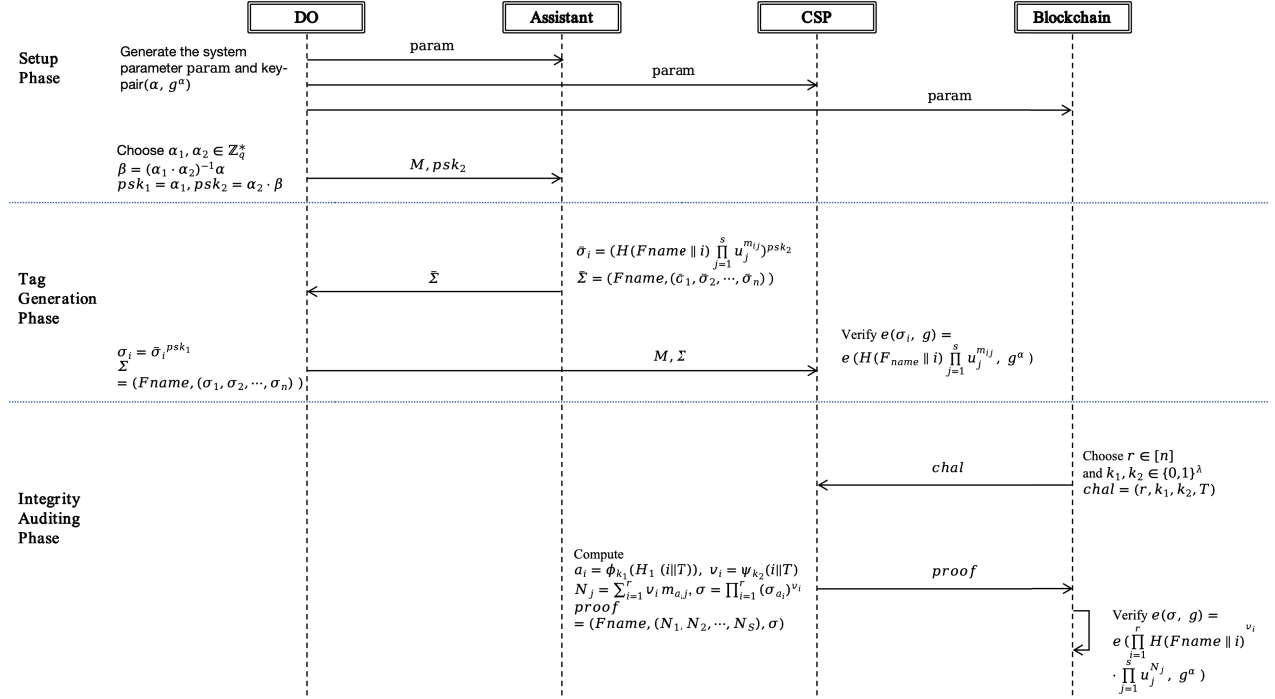


Fig. 2. Interactions among different entities.

- 4) *Forgery*: After obtaining the received challenge chal , $\mathcal{A}$ generates a proof proof' corresponding with a forged data file M' , and returns it to $\mathcal{C}$. If

$$\text{Verify}(\text{chal}, \text{proof}') = 1$$

then $\mathcal{A}$ is considered to have won the security game.

Definition 2: If for any PPT adversary $\mathcal{A}$, the probability of $\mathcal{A}$ winning Game 2 is negligible, then we call the integrity auditing scheme sound against the semi-honest CSP.

V. OUR PROPOSED CONSTRUCTION

In this section, we will introduce the concrete construction of our proposed system. In particular, the descriptions of the algorithms are given as follows, and the interactions between the entities are shown in Fig. 2.

A. Setup Phase

- 1) $\text{Setup}(1^\lambda) \rightarrow (\text{param}, \text{msk})$: Given λ , this algorithm first generates two multiplicative cyclic groups G_1, G_2 with the same prime order q , and defines a bilinear map $e: G_1 \times G_1 \rightarrow G_2$. Then randomly select s elements $u_1, \dots, u_s$ from G_1 and define $H: \{0, 1\}^* \rightarrow G_1$, $H_1: \{0, 1\}^* \rightarrow [n]$ as a hash function, where n is number of data blocks for each file. Let $\phi: \{0, 1\}^\lambda \times [n] \rightarrow [n]$ and $\psi: \{0, 1\}^\lambda \times \{0, 1\}^* \rightarrow \mathbb{Z}_q$ be PRP and PRF, respectively. Next, randomly choose $\alpha \in \mathbb{Z}_q^*$ as the master secret key and compute $\text{pk} = g^\alpha \in G_1$, where g is a generator of G_1 . Finally, output

$$\text{param} = (q, G_1, G_2, e, n, \{u_1, \dots, u_s\}, H, H_1, \phi, \psi, \text{pk})$$

and $\text{msk} = \alpha$ as the system public parameter and master secret key, respectively.

- 2) $\text{KeyGen}(\text{msk}) \rightarrow (\text{psk}_1, \text{psk}_2)$: Given $\text{msk} = \alpha$, this algorithm randomly chooses α_1, α_2 from $\mathbb{Z}_q^*$ and computes $\beta = (\alpha_1 \cdot \alpha_2)^{-1} \alpha$. Then define $\text{psk}_1 = \alpha_1$, $\text{psk}_2 = \alpha_2 \cdot \beta$, and output them. Note that psk_2 is given to the Assistant and psk_1 is kept secret by DO.

B. Tag Generation Phase

- 1) $\text{AssistTag}(M, \text{psk}_2) \rightarrow \bar{\Sigma}$: First, randomly choose a file name $\text{Fname} \in \{0, 1\}^*$ for the file M . Parse M into n blocks $\{M_1, M_2, \dots, M_n\}$, and each block has s sectors: m_{ij} , where $1 \leq i \leq n$ and $1 \leq j \leq s$. For each M_i , compute

$$\bar{\sigma}_i = \left(H(\text{Fname} \parallel i) \cdot \prod_{j=1}^s u_j^{m_{ij}} \right)^{\text{psk}_2}.$$

Define $\bar{\Sigma} = (\text{Fname}, (\bar{\sigma}_1, \bar{\sigma}_2, \dots, \bar{\sigma}_n))$ and output it.

- 2) $\text{Tag}(\bar{\Sigma}, \text{psk}_1) \rightarrow \Sigma$. Based on the private psk_1 , DO computes

$$\sigma_i = (\bar{\sigma}_i)^{\text{psk}_1}$$

for $1 \leq i \leq n$. Define $\Sigma = (\text{Fname}, (\sigma_1, \sigma_2, \dots, \sigma_n))$ and output it.

- 3) $\text{VeriTag}(M, \Sigma) \rightarrow 1/0$. For the received (M, Σ) , CSP checks its correctness by verifying the following equations:

$$e(\sigma_i, g) = e \left(H(\text{Fname} \parallel i) \cdot \prod_{j=1}^s u_j^{m_{ij}}, \text{pk} \right) \quad 1 \leq i \leq n.$$

If all of them hold, output 1 and store (M, Σ) . Otherwise, output 0 and return the result to DO.

C. Integrity Auditing Phase

- 1) **ChalGen** $\rightarrow$ **chal**: When Blockchain receives the audit request from DO, it randomly chooses $r \in [n]$ as the number of challenged blocks, and $k_1, k_2 \in \{0, 1\}^l$ as the keys of PRP and PRF. Then record the current time as T . Define $\text{chal} = (r, k_1, k_2, T)$ and give it to CSP. Note that chal is also recorded on the Blockchain for transparency of the audit process.
- 2) **ProofGen**(chal, M, Σ) $\rightarrow$ *Proof*: After receiving the challenge message chal , CSP first computes

$$a_i = \phi_{k_1}(H_1(i||T)) \in [n], v_i = \psi_{k_2}(i||T) \in Z_q \quad (1)$$

for $1 \leq i \leq r$. For $1 \leq j \leq s$, define

$$N_j = \sum_{i=1}^r v_i m_{a_i j}, \quad \sigma = \prod_{i=1}^r (\sigma_{a_i})^{v_i}.$$

Finally, define $\text{proof} = (\text{Fname}, (N_1, N_2, \dots, N_s), \sigma)$ as the proof returned to Blockchain.

- 3) **Verify**($\text{chal}, \text{proof}$) $\rightarrow$ 1/0. After receiving the returned proof, Blockchain computes a_i, v_i from (1). Then verify if the following equation holds:

$$e(\sigma, g) = e\left(\prod_{i=1}^r H(\text{Fname}||a_i)^{v_i} \cdot \prod_{j=1}^s u_j^{N_j}, \text{pk}\right). \quad (2)$$

If it is, output 1. Else output 0. Note that the audit result will also be recorded on the Blockchain.

D. Revocation and Replacement of Assistants

- 1) *Revocation of Assistant*: If DO wants to revoke the current Assistant, then it can perform the following steps. Let ID and psk_2 be its identity and partial secret key. Compute $\text{pk}_{\text{ID}} = g^{\text{psk}_2} \in G_1$ and record $(\text{ID}, *, \text{pk}_{\text{ID}}, t, \text{invalid})$ in a list L , where t denotes the time of revocation.
- 2) *Replacement of Assistant*: If DO wants to reassign a new Assistant to perform the generation of partial tag, then it randomly chooses α'_1, α'_2 from Z_q , and computes $\beta' = (\alpha'_1 \cdot \alpha'_2)^{-1} \alpha$. Define $\text{psk}'_1 = \alpha'_1$, and $\text{psk}'_2 = \alpha'_2 \cdot \beta$. Give psk'_2 to the new Assistant with identity ID' and record $(\text{ID}', \text{psk}'_2, \text{pk}_{\text{ID}'}, t', \text{valid})$ in L , where t' denotes the time of replacement.

VI. CORRECTNESS AND SECURITY ANALYSIS

A. Correctness Analysis

We analyze the correctness of our scheme in tag verification and integrity auditing.

In the **VeriTag** algorithm, if Assistant and DO correctly generate the authentication tag to the file M , the tag set Σ can pass the verification of CSP, which is, the equation

$e(\sigma_i, g) = e\left(H(\text{Fname}||i) \cdot \prod_{j=1}^s u_j^{m_{ij}}, \text{pk}\right)$, for $1 \leq i \leq n$ holds, as specified below

$$\begin{aligned} e(\sigma_i, g) &= e\left(\left(H(\text{Fname}||i) \cdot \prod_{j=1}^s u_j^{m_{ij}}\right)^\alpha, g\right) \\ &= e\left(H(\text{Fname}||i) \cdot \prod_{j=1}^s u_j^{m_{ij}}, \text{pk}\right). \end{aligned}$$

In the **Verify** algorithm, if CSP correctly stores the file with the filename F_{id} , the proof can pass the integrity verification of Blockchain, which is, the equation $e(\sigma, g) = e\left(\prod_{i=1}^r H(\text{Fname}||a_i)^{v_i} \cdot \prod_{j=1}^s u_j^{N_j}, \text{pk}\right)$ holds, as specified below

$$\begin{aligned} e(\sigma, g) &= e\left(\prod_{i=1}^r \sigma_{a_i}^{v_i}, g\right) \\ &= e\left(\prod_{i=1}^r \left(H(\text{Fname}||i) \cdot \prod_{j=1}^s u_j^{m_{a_i j}}\right)^{\alpha v_i}, g\right) \\ &= e\left(\prod_{i=1}^r H_1(\text{Fname}||i)^{\alpha \cdot v_i}, g\right) \\ &\quad \cdot e\left(\left(\prod_{j=1}^s u_j^{\sum_{i=1}^r v_i \cdot m_{a_i j}}\right)^\alpha, g\right) \\ &= e\left(\prod_{i=1}^r H_1(\text{Fname}||i)^{v_i}, g^\alpha\right) \cdot e\left(\prod_{j=1}^s u_j^{N_j}, g^\alpha\right) \\ &= e\left(\prod_{i=1}^r H(\text{Fname}||i)^{v_i} \cdot \prod_{j=1}^s u_j^{N_j}, \text{pk}\right). \end{aligned}$$

B. Security Analysis

Theorem 1: In the proposed scheme, if the hardness assumption of the computational Diffie–Hellman (CDH) problem holds, then it is difficult for the Assistant to forge a valid tag that can pass the verification by CSP.

Proof: If the PPT adversary $\mathcal{A}$ wins **Game 1** with a nonnegligible probability, then we can build a simulator $\mathcal{S}$ that can use the capabilities of $\mathcal{A}$ to solve the CDH problem. Given a multiplicative cyclic group $G_1 = \langle g \rangle$ of prime order q and a tuple (g, g^a, g^b) , $\mathcal{S}$'s goal is to compute g^{ab} .

- 1) *Setup*: For each $1 \leq j \leq s$, $\mathcal{S}$ randomly selects $\alpha_j, \beta_j \in Z_q^*$ and sets $u_j = g^{\alpha_j} \cdot (g^b)^{\beta_j}$, psk_1 then generates the system parameter $\text{param} = (q, G_1, G_2, e, \{u_1, \dots, u_s\}, \text{pk} = g^a)$. $\mathcal{S}$ sends param to $\mathcal{A}$.
- 2) *Queries*: $\mathcal{A}$ can request the following query.

- a) *Key-Query*: $\mathcal{S}$ randomly choose $\text{psk} \in Z_q^*$ and sends psk to $\mathcal{A}$.
- b) *Hash-Query*: $\mathcal{A}$ can perform Hash-Query using an adaptively selected Fname . For each $1 \leq i \leq n$, $\mathcal{S}$ randomly selects $r_i \in Z_q^*$, then defines

$$H(\text{Fname}||i)$$

$$= g^{r_i} / \left(g^{\sum_{j=1}^s \alpha_j m_{ij}} \cdot (g^b)^{\sum_{j=1}^s \beta_j m_{ij}} \right).$$

- c) *Tag-Query*: $\mathcal{A}$ can send file blocks $M_i, 1 \leq i \leq n$ to $\mathcal{S}$ and computes assist tag

$$\bar{\sigma}_i = \left(H(\text{Fname}||i) \cdot \prod_{j=1}^s u_j^{m_{ij}} \right)^{\text{psk}}$$

to $\mathcal{S}$ for *Tag-Query*. $\mathcal{S}$ calculates the tag as follows:

$$\begin{aligned} \sigma_i &= \left(H(\text{Fname}||i) \cdot \prod_{j=1}^s u_j^{m_{ij}} \right)^a \\ &= \left(g^{r_i} / \left(g^{\sum_{j=1}^s \alpha_j m_{ij}} \cdot (g^b)^{\sum_{j=1}^s \beta_j m_{ij}} \right) \right. \\ &\quad \cdot \left. \left(\prod_{j=1}^s \left(g^{\alpha_j} \cdot (g^b)^{\beta_j} \right)^{m_{ij}} \right) \right)^a \\ &= \left(g^{r_i} / \left(g^{\sum_{j=1}^s \alpha_j m_{ij}} \cdot (g^b)^{\sum_{j=1}^s \beta_j m_{ij}} \right) \right. \\ &\quad \cdot \left. \left(g^{\sum_{j=1}^s \alpha_j m_{i,j}} \cdot (g^b)^{\sum_{j=1}^s \beta_j m_{i,j}} \right) \right)^a \\ &= (g^a)^{r_i}. \end{aligned}$$

- 3) *Forgery*: $\mathcal{A}$ forges a tag Σ' for a data file M' not be queried. If $\mathcal{A}$ succeeds in generating a verifiable tag set Σ' , then the following equations hold:

$$e(\sigma'_i, g) = e \left(H(\text{Fname}||i) \cdot \prod_{j=1}^s u_j^{m'_{ij}}, g^a \right) \quad 1 \leq i \leq n \quad (3)$$

and

$$e(\sigma_i, g) = e \left(H(\text{Fname}||i) \cdot \prod_{j=1}^s u_j^{m_{ij}}, g^a \right) \quad 1 \leq i \leq n. \quad (4)$$

Let $\Delta m_{ij} = m'_{ij} - m_{ij}$, at least one Δm_{ij} that does not equal to 0, otherwise M' is be queried. Dividing the verification equation (3) by the verification equation (4) can obtain

$$\begin{aligned} e(\sigma'_i / \sigma_i, g) &= e \left(\prod_{j=1}^s u_j^{\Delta m_{ij}}, g^a \right) \\ &= e \left(\prod_{j=1}^s \left(g^{\alpha_j} \cdot (g^b)^{\beta_j} \right)^{\Delta m_{ij}}, g^a \right) \\ &= e \left(g^{\sum_{j=1}^s \alpha_j \Delta m_{ij}} \cdot (g^b)^{\sum_{j=1}^s \beta_j \Delta m_{ij}}, g^a \right). \end{aligned}$$

Based on this formula, $\mathcal{S}$ can go further and get

$$\begin{aligned} e \left((\sigma'_i / \sigma_i) \cdot (g^a)^{-\sum_{j=1}^s \alpha_j \Delta m_{ij}}, g \right) \\ = e \left(g^b, g^a \right)^{\sum_{j=1}^s \beta_j \Delta m_{ij}}. \end{aligned}$$

Finally, $\mathcal{S}$ can get the solution

$$g^{ab} = \left((\sigma'_i / \sigma_i) \cdot (g^a)^{-\sum_{j=1}^s \alpha_j \Delta m_{ij}} \right)^{\frac{1}{\sum_{j=1}^s \beta_j \Delta m_{ij}}}$$

for the CDH problem.

From the above formula, we can see that the CDH problem cannot be solved if and only if $\sum_{j=1}^s \beta_j \Delta m_{ij} = 0$. Since $\Delta m_{ij} \neq 0, 1 \leq j \leq s$ exists and $\beta_j, 1 \leq j \leq s$ are random values, $\Pr \left[\sum_{j=1}^s \beta_j \Delta m_{ij} = 0 \right] = 1/q$. This means that $\mathcal{A}$ is able to solve the CDH problem with a large probability of $1 - 1/q$, which contradicts the hardness assumption of the CDH problem. Therefore, it is impossible for $\mathcal{A}$ to win Game 1 with a nonnegligible probability.

Theorem 2: In the proposed scheme, if the hardness assumption of the CDH problem holds, then the proposed integrity auditing scheme is secure.

Proof: If the PPT adversary $\mathcal{A}$ wins Game 2 with a nonnegligible probability, then we can build a simulator $\mathcal{S}$ to solve the CDH problem.

- 1) *Setup*: For each $1 \leq j \leq s$, $\mathcal{S}$ randomly selects $\alpha_j, \beta_j \in \mathbb{Z}_q^*$ and sets $u_j = g^{\alpha_j} \cdot (g^b)^{\beta_j}$, then generates the system parameter

$$\text{param} = (q, G_1, G_2, e, \{u_1, \dots, u_s\}, \text{pk} = g^a).$$

$\mathcal{S}$ sends param to $\mathcal{A}$.

- 2) *Queries*: $\mathcal{A}$ can repeat the following query.

- a) *Hash-Query*: $\mathcal{A}$ can perform Hash-Query using an adaptively selected Fname. For each $1 \leq i \leq n$, $\mathcal{S}$ randomly selects $r_i \in \mathbb{Z}_q^*$, then defines

$$\begin{aligned} H(\text{Fname}||i) \\ = g^{r_i} / \left(g^{\sum_{j=1}^s \alpha_j m_{ij}} \cdot (g^b)^{\sum_{j=1}^s \beta_j m_{ij}} \right). \end{aligned}$$

- b) *Tag-Query*: $\mathcal{A}$ can adaptively submit M to $\mathcal{S}$ for Tag-Query. $\mathcal{S}$ gets the following formula:

$$\begin{aligned} \prod_{j=1}^s u_j^{m_{ij}} &= \prod_{j=1}^s \left(g^{\alpha_j} \cdot (g^b)^{\beta_j} \right)^{m_{ij}} \\ &= g^{\sum_{j=1}^s \alpha_j m_{i,j}} \cdot (g^b)^{\sum_{j=1}^s \beta_j m_{i,j}}. \end{aligned}$$

$\mathcal{S}$ calculates the tag set $\Sigma = (\sigma_1, \sigma_2, \dots, \sigma_n)$, specifically as follows:

$$\begin{aligned} \sigma_i &= \left(H(\text{Fname}||i) \cdot \prod_{j=1}^s u_j^{m_{ij}} \right)^a \\ &= \left(g^{r_i} / \left(g^{\sum_{j=1}^s \alpha_j m_{ij}} \cdot (g^b)^{\sum_{j=1}^s \beta_j m_{ij}} \right) \right. \\ &\quad \cdot \left. \left(g^{\sum_{j=1}^s \alpha_j m_{i,j}} \cdot (g^b)^{\sum_{j=1}^s \beta_j m_{i,j}} \right) \right)^a \\ &= (g^a)^{r_i}. \end{aligned}$$

- 3) *Challenge*: $\mathcal{S}$ randomly chooses $a_i \in \mathbb{Z}_n^*, v_i \in \mathbb{Z}_q^*$ for $1 \leq i \leq r$ and sends the challenge $\text{chal} = \{a_i, v_i\}$ to $\mathcal{A}$. After receiving the challenge message, $\mathcal{A}$ generates the corresponding proof $\text{proof} = (\text{Fname}, (N_1, N_2, \dots, N_s), \sigma)$ and returns it to $\mathcal{S}$.

- 4) *Forgery*: After obtaining the received challenge chal , $\mathcal{A}$ forges and returns the corresponding proof $\text{proof}' = (\text{Fname}, (N'_1, N'_2, \dots, N'_s), \sigma')$ to $\mathcal{S}$. If $\mathcal{A}$ succeeds in forging a proof, then the following equations hold:

$$e(\sigma', g) = e\left(\prod_{i \in K} H(\text{Fname} \| a_i)^{v_i} \cdot \prod_{j=1}^s u_j^{N'_j}, g^a\right) \quad (5)$$

and

$$e(\sigma, g) = e\left(\prod_{i \in K} H(\text{Fname} \| a_i)^{v_i} \cdot \prod_{j=1}^s u_j^{N_j}, g^a\right). \quad (6)$$

Let $\Delta N_j = N'_j - N_j$, assuming there is at least one ΔN_j that does not equal 0. Dividing the verification equation (5) by the verification equation (6) can obtain

$$\begin{aligned} e(\sigma'/\sigma, g) &= e\left(\prod_{j=1}^s u_j^{\Delta N_j}, g^a\right) \\ &= e\left(\prod_{j=1}^s (g^{\alpha_j} \cdot (g^b)^{\beta_j})^{\Delta N_j}, g^a\right) \\ &= e\left(g^{\sum_{j=1}^s \alpha_j \Delta N_j} \cdot (g^b)^{\sum_{j=1}^s \beta_j \Delta N_j}, g^a\right). \end{aligned}$$

Based on this formula, $\mathcal{S}$ can go further and get

$$\begin{aligned} e((\sigma'/\sigma) \cdot (g^a)^{-\sum_{j=1}^s \alpha_j \Delta N_j}, g) \\ = e(g^b, g^a)^{\sum_{j=1}^s \beta_j \Delta N_j}. \end{aligned}$$

Finally, $\mathcal{S}$ can get the solution

$$g^{ab} = ((\sigma'/\sigma) \cdot (g^a)^{-\sum_{j=1}^s \alpha_j \Delta N_j})^{\frac{1}{\sum_{j=1}^s \beta_j \Delta N_j}}$$

for the given CDH instance.

From the above formula, we can see that the CDH problem cannot be solved if and only if $\sum_{j=1}^s \beta_j \Delta N_j = 0$. Since $\Delta N_j \neq 0$, $1 \leq j \leq s$ exists and β_j , $1 \leq j \leq s$ are random values, $\Pr\left[\sum_{j=1}^s \beta_j \Delta N_j = 0\right] = 1/q$. This means that $\mathcal{A}$ is able to solve the CDH problem with a large probability of $1 - 1/q$, which contradicts the hardness assumption of the CDH problem. Therefore, it is impossible for $\mathcal{A}$ to win Game 2 with a nonnegligible probability. To sum up, we have completed the proof that a valid audit proof generated by CSP can pass the integrity auditing only if there is real data stored in it.

Remark 1: In the proposed scheme, it is difficult for Blockchain to recover the data file. That is to say, the proposed scheme can achieve privacy protection.

Heuristic Analysis: In our proposed scheme, owing to the use of PRP, PRF, and timestamps, Blockchain cannot always choose the same data file blocks as challenged blocks in the integrity auditing phase to compute specific data file blocks.

The proposed scheme uses PRP ϕ , PRF ψ , and timestamps T to generate a challenge message. The Blockchain chooses the number $r \in [n]$ as challenge data blocks. If the Blockchain wants to recover the data of the data file block, it needs to get the same challenge data block N_j for r times. However, the element $m_{a_{ij}}$ of N_j is selected by PRP ϕ , the probability of success is $(1/n^2)$. Consequently, our proposed scheme can achieve privacy protection.

VII. PERFORMANCE ANALYSIS

In this section, we theoretically compare the proposed scheme with the schemes of Wang et al. [24], Hu et al. [25], and Huang et al. [30] in terms of storage cost, communication cost, and computational cost. These schemes also focus on data sharing and integrity auditing in cloud storage, but they adopt different strategies for tag generation. Specifically, in [24] and [30], the DO directly performs the tag computation, whereas in [25], the assistant is responsible for generating the signatures. Therefore, these schemes are highly comparable to ours. Furthermore, corresponding experiments are conducted to validate the advantages of the proposed scheme.

For simplicity, the relevant symbols are defined in Nomenclature with their corresponding values. Specifically, we set $|G_1| = 512$ bits and $|Z_q| = |Z_q^*| = 160$ bits. In our scheme, each data block can be divided into multiple sectors to reduce storage overhead. Nevertheless, for simplicity and fairness, only the case where a file consists of n data blocks is considered, without further dividing each block into s sectors, i.e., $s = 1$.

A. Computation Cost

In the *Tag Generation* phase, to generate authentication tags for the DO's data file, the Assistant (AS) and the DO will execute the algorithms *AssistTag* and *Tag*, respectively. The computation for AS is expressed as

$$\tilde{\sigma}_i = \left(H(F(\text{name}) \| i) \cdot \prod_{j=1}^s u_j^{m_{ij}} \right)^{\text{psk}_2}$$

where $1 \leq i \leq n$. The computation cost of *AssistTag* in AS is $n(T_H + (s + 1) \cdot T_{\text{exp}} + s \cdot T_{\text{mul}})$. The computation of DO is the formula $\sigma_i = (\tilde{\sigma}_i)^{\text{psk}_1}$, where $1 \leq i \leq n$. Therefore, the computational cost of DO is $n \cdot T_{\text{exp}}$. In the **Integrity Auditing** phase, to compute the integrity proof, the CSP runs the algorithm *ProofGen*, which needs

$$\begin{aligned} s((r - 1) \cdot T_{Z_{qa}} + r \cdot T_{Z_{qm}}) + r(T_{\text{exp}} + T_{\text{prp}} + T_{\text{prf}}) \\ + (r - 1) \cdot T_{\text{mul}}. \end{aligned}$$

It is worth mentioning that the computation costs of T_{prp} and T_{prf} are negligible. And in the session of verification, the Blockchain (BC) runs algorithm *Verify*, in which the computation cost is

$$\begin{aligned} 2T_{\text{pair}} + (r + s)T_{\text{exp}} + (r + s - 1)T_{\text{mul}} \\ + r(T_{\text{prp}} + T_{\text{prf}} + T_H). \end{aligned}$$

The same approach is adopted to evaluate the computational costs of the schemes in [24], [25], and [30] for the *Tag Generation*, *Proof Generation*, and *Verify* phases. The comparison results are summarized in Table I, where we set $s = 1$, and V_{IBS} denotes the computational cost of a single IBS verification in the scheme proposed in [25].

From the comparison, it is clear that in the *Tag Generation* stage, the computational cost of our scheme is higher than that of [25] and [30], but lower than that of [24]. Since both our scheme and [25] delegate the task of tag generation to

TABLE I
COMPUTATION COST COMPARISON

	Our scheme	[24]	[25]	[30]
Tag Generation	$n(T_H + 3T_{exp} + T_{mul})$	$3n(T_H + T_{exp}) + 2n \cdot T_{mul}$	$n(T_H + 2T_{exp} + 2T_{mul})$	$n(T_H + 2T_{exp} + 2T_{mul})$
ProofGen	$(r-1)T_{Z_{qa}} + r \cdot T_{Z_{qm}} + r \cdot T_{exp} + (r-1)T_{mul}$	$r \cdot (T_{Z_{qa}} + T_{Z_{qm}} + T_{exp}) + (2r-1)T_{mul}$	$(r-1)T_{Z_{qa}} + r \cdot T_{Z_{qm}} + r \cdot T_{exp} + (r-1)T_{mul}$	$(r-1)T_{Z_{qa}} + r \cdot T_{Z_{qm}} + r \cdot T_{exp} + (r-1)T_{mul}$
Verify	$2T_{pair} + (r+1)T_{exp} + rT_{mul} + r \cdot T_H$	$2T_{pair} + (r+2)T_{exp} + (r+1)T_{mul} + (r+1)T_H$	$3T_{pair} + (r+2)T_{exp} + (r+1)T_{mul} + (r+1)T_H + (r-1)T_{Z_{qa}} + V_{IBS}$	$3T_{pair} + (r+2)T_{exp} + (r+1)T_{mul} + (r+1)T_H + (r-1)T_{Z_{qa}}$

TABLE II
COMMUNICATION COST COMPARISON

	Our scheme(bits)	[24](bits)	[25](bits)	[30](bits)
AS to DO	$n \cdot G_1 = 512n$	-	-	-
DO/AS to CSP	$n \cdot Z_q + n \cdot G_1 = 672n$	$n \cdot Z_q + n \cdot G_1 = 672n$	$(n+1) \cdot Z_q + (n+2) G_1 = 1184 + 672n$	$n \cdot Z_q + n \cdot G_1 = 672n$
BC/TPA to CSP	$4 Z_q = 640$	$4 Z_q = 640$	$2r \cdot Z_q = 320r$	$2r \cdot Z_q = 320r$
CSP to BC/TPA	$ G_1 + 2 Z_q = 832$	$ Z_q + G_1 = 672$	$2 Z_q + 2 G_1 = 1344$	$ Z_q + G_1 = 672$
Total Cost	$(2n+1) G_1 + (n+6) Z_q = 1472 + 1184n$	$(n+5) Z_q + (n+1) G_1 = 1312 + 672n$	$(n+4) G_1 + (n+3+2r) Z_q = 2528 + 672n + 320r$	$(n+1) G_1 + (n+1+2r) Z_q = 672 + 672n + 320r$

the assistant (AS), the computational burden on the DO is alleviated, whereas in [30] the DO is required to generate the tags. The slightly higher computational cost compared with [25] and [30] is acceptable, as our scheme requires DO confirmation in the tag generation process to produce the final tag, thereby enhancing the overall security of the system. In the *Proof Generation* stage, the computational cost of our scheme is the same as that of [25] and [30], and lower than that of [24]. In the *Verify* stage, the computational cost of our scheme is also lower than that of [24], [25], and [30].

B. Communication Cost

According to the proposed scheme in this article, the communication cost is divided into four main components: 1) the Assistant (AS) sends partial authentication tags $\bar{\sigma}_i$ for $1 \leq i \leq n$ to the DO, which costs $n \cdot |G_1|$, where $|G_1|$ is the length of G_1 ; 2) the DO sends file blocks and the corresponding tags σ_i for $1 \leq i \leq n$ to the CSP, which costs $n \cdot s \cdot |Z_q| + n \cdot |G_1|$; 3) the Blockchain sends an audit challenge $\text{chal} = \{r, k_1, k_2, T\}$ to the CSP, which costs $4|Z_q|$; and 4) The CSP returns the audit proof

$$\text{proof} = (\text{Fname}, (N_1, N_2, \dots, N_s), \sigma)$$

to the Blockchain, which costs $|G_1| + (s+1)|Z_q|$. Therefore, the total communication cost is $(2n+1)|G_1| + (n+6)|Z_q|$. AS to DO, DO/AS to CSP, BC/TPA to CSP, and CSP to BC/TPA represent the communication costs for these four components, respectively.

Similarly, the communication costs for the schemes in [24], [25], and [30] can also be obtained using the above method. A comparison of the communication costs between the schemes is shown in Table II, where we set $s = 1$.

It can be observed that in the *AS to DO* component, the communication cost of our scheme is higher than that of [24], [25], and [30], because in our scheme the AS needs to send partial authentication tags to the DO. In the *DO/AS to CSP*

component, the communication cost of our scheme is the same as that of [24], and lower than that of [25] and [30]. Similarly, in the *BC/TPA to CSP* component, as the number of challenge blocks r increases, the communication cost of our scheme remains the same as that of [24], and lower than that of [25] and [30]. In addition, in [25] and [30], the communication cost increases linearly with the number of challenge blocks. Finally, in the *CSP to BC/TPA* component, the communication cost of our scheme is higher than that of [24] and [30], but lower than that of [25].

C. Storage Cost

In the proposed scheme, the storage overhead is primarily distributed across four entities: 1) *DO*: the DO deletes the local copies after transferring the entire file and corresponding authentication tags to the CSP. Thus, the DO only stores its partial secret key, and the storage cost of the DO is $|Z_q^*|$; 2) *AS*: the AS also deletes the file after computing the partial authentication tags. Therefore, the storage cost of the AS is $|Z_q^*|$; 3) *CSP*: the CSP stores the file blocks M_i for $1 \leq i \leq n$ and the tags σ_i for $1 \leq i \leq n$, which incur a storage cost of $n(|G_1| + s \cdot |Z_q|)$; and 4) *BC/TPA*: the Blockchain only needs to store the challenge message, with a storage cost no greater than $4|Z_q|$. Therefore, the total storage cost across the four entities is $n \cdot |G_1| + (n+6)|Z_q|$.

Similarly, the storage costs for the schemes in [24], [25], and [30] across the relevant entities can be derived, where s^* represents the number of sensitive blocks in [25]. For the sake of comparison and fairness, $|Z_q^*|$ is approximated as $|Z_q|$. A comparison of the storage costs between the schemes is shown in Table III, where we set $s = 1$.

From the comparison, it can be observed that the total storage cost of our scheme is comparable to that of [24] and lower than those of [25] and [30] across all four entities. Although an Assistant (AS) is introduced, the AS only stores its own secret key, which incurs negligible overhead. Overall,

TABLE III
STORAGE COST COMPARISON

	Our scheme(bits)	[24](bits)	[25](bits)	[30](bits)
DO	$ Z_q = 160$	$2 G_1 = 1024$	$ G_1 + s^* \cdot Z_q = 512 + 160s^*$	$ G_1 + Z_q = 672$
AS	$ Z_q = 160$	-	$ G_1 = 512$	-
CSP	$n(G_1 + Z_q) = 672n$	$n(G_1 + Z_q) = 672n$	$(n+2) G_1 + (n+1) Z_q = 1184 + 672n$	$n(G_1 + Z_q) = 672n$
BC/TPA	$4 Z_q = 640$	$4 Z_q = 640$	$2r \cdot Z_q = 320r$	$2r \cdot Z_q = 320r$
Total Cost	$n \cdot G_1 + (n+6) Z_q = 960 + 672n$	$(n+2) G_1 + (n+4) Z_q = 1664 + 672n$	$(n+4) G_1 + (n+1+2r) Z_q = 2208 + 672n + 160s^* + 320r$	$(n+1) G_1 + (n+1+2r) Z_q = 672 + 672n + 320r$

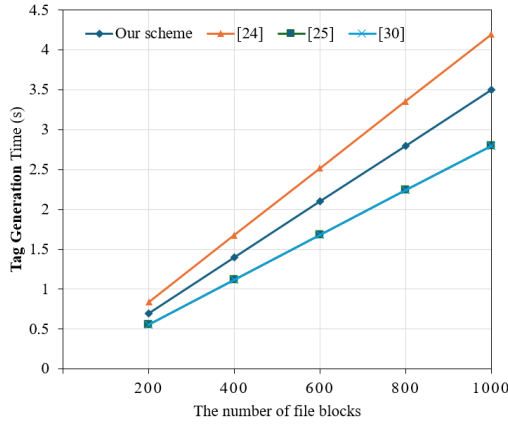


Fig. 3. Comparison of time overhead in tag generation.

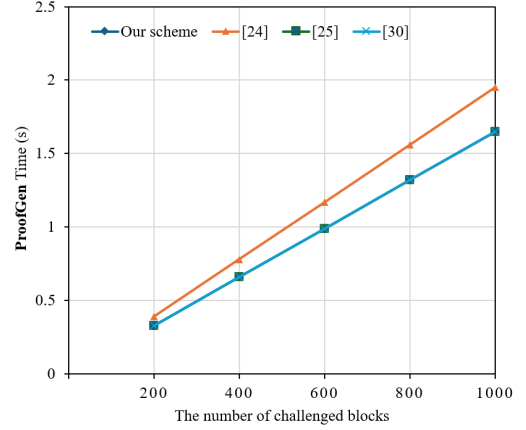


Fig. 4. Comparison of time overhead in proof generation.

our scheme demonstrates favorable performance in terms of storage cost.

D. Experimental Analysis

To further evaluate the computational efficiency of the proposed scheme, it was implemented in C using the pairing-based cryptography (PBC) library for bilinear pairing operations and the OpenSSL cryptographic library for standard cryptographic functions. Experimental evaluations were conducted to assess the performance of the scheme and to compare it with the schemes in [24] and [25]. For the experiments, the file shared in the cloud was set to a size of 20 KB, divided into $n = 1000$ blocks, with each block being 160 bits in size. The number of sensitive blocks in [25] was set to 0, i.e., $s^* = 0$. The configurations used in the experiments are as follows: 1) *Host Machine*: Windows 10 operating system, with an Intel(R) Core(TM) i7-8750H CPU @ 2.20 GHz and 8 GB of RAM and 2) *Virtual Machine*: configured using VMware Workstation Pro with Ubuntu 18.04.2 as the guest OS, allocated with 4 GB of RAM, 42 GB of disk space, and two dual-core CPUs (total of 4 virtual processors).

Figs. 3–5 present a comparison of the computational time of our scheme with those of Wang et al. [24], Hu et al. [25], and Huang et al. [30] in the *Tag Generation*, *Proof Generation*, and *Verify* stages. Figs. 6 and 7 show the comparison in terms of total communication cost and total storage cost, respectively.

1) *Tag Generation Cost*: In this experiment, the tag-generation time depends on the number of blocks in the cloud-shared data file. The file is partitioned into 1000 data blocks; therefore, we generate authentication tags for

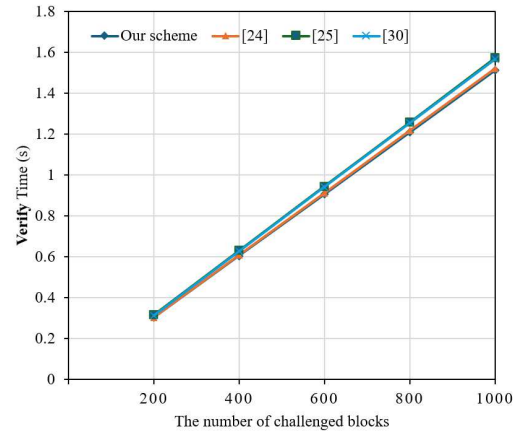


Fig. 5. Comparison of time overhead in proof verification.

configurations where the number of blocks ranges from 200 to 1000. In Fig. 3, when the file is partitioned into 200 blocks, the tag-generation times of our scheme and those of [24], [25], and [30] are 0.7014, 0.8424, 0.5634, and 0.5629 s, respectively; when partitioned into 1000 blocks, these times increase to 3.507, 4.212, 2.8155, and 2.8161 s, respectively. Consequently, in the *Tag Generation* stage, our scheme incurs a higher computational cost than [25] and [30], but a lower cost than [24]. The reason is that, in our scheme, the DO performs only a lightweight computation to finalize the authentication tags from the partial tags produced by the AS, whereas in [24] and [30] the DO performs the entire tag-generation computation. Although [25] also delegates tag generation to the AS, its design lets the AS output the

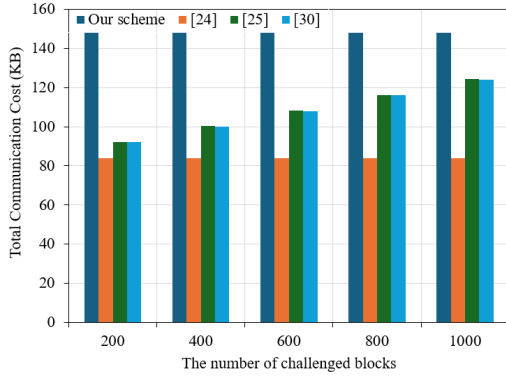


Fig. 6. Comparison of total communication cost.

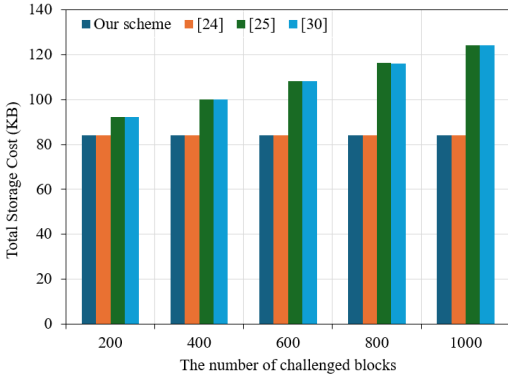


Fig. 7. Comparison of total storage cost.

final, cloud-uploadable tag without a DO-side finalization step. This leads to the following differences: 1) a larger trust boundary—once the AS is compromised or colludes with the CSP, it may bypass the DO; 2) weaker unforgeability—an attacker only needs to compromise the AS-side environment to forge final tags, while in our scheme no valid tag can be produced without the DO’s finalization; 3) inferior auditability and accountability—lacking the two-step “partial → final” trail makes anomaly localization harder; and 4) insufficient privacy/compliance control—there is no point at which the DO explicitly approves finalization. In contrast, our two-step design adds only a small amount of lightweight computation at the DO, yet significantly improves security and controllability without increasing the costs of *Proof Generation* and *Verify*.

2) *Proof Generation Cost*: Upon receiving a challenge message from the blockchain or the TPA, the cloud computes an audit proof for the shared file. The computational cost of generating the audit proof depends on the number of challenged blocks. In this experiment, we set the number of challenged blocks from 200 to 1000. As shown in Fig. 4, when the number of challenged blocks is 200, the audit-proof generation times of our scheme and those of [24], [25], and [30] are 0.3315, 0.3912, 0.3312, and 0.3321 s, respectively; when the number increases to 1000, these times increase to 1.6582, 1.9563, 1.6585, and 1.6574 s, respectively. At the same time, the computational cost of our scheme is the same as that of [25] and [30], and lower than that of [24]. This is because our scheme, together with [25] and [30], adopts the

same algorithm to generate audit proofs, whereas [24] incurs additional overhead due to the use of auxiliary tag aggregation in the proof-generation process.

3) *Verify Cost*: In the *Verify* stage, the audit proof returned by the cloud is verified to ensure the integrity of the shared data. According to the verification algorithm and equations, the computational cost is mainly linear with respect to the number of challenged blocks. We set the number of challenged blocks from 200 to 1000 to observe its impact on the verification overhead. As shown in Fig. 5, when the number of challenged blocks is 200, the verification times of our scheme and those of [24], [25], and [30] are 0.302, 0.304, 0.3144, and 0.3134 s, respectively. When the number of challenged blocks increases to 1000, these times increase to 1.510, 1.520, 1.572, and 1.567 s, respectively. The computational cost of our scheme in this stage is slightly lower than that of [24], [25], and [30], since our verification process involves fewer exponentiation and multiplication operations.

4) *Communication Cost*: The communication cost is mainly determined by the interactions among different entities in the scheme. As illustrated in Fig. 6, when the cloud-stored file is divided into 1000 blocks and the number of challenged blocks ranges from 200 to 1000, the total communication costs of our scheme and [24] remain constant at 148 and 84 KB, respectively. In contrast, the total communication cost of [25] increases from 92.232–124.232 KB, and that of [30] increases from 92–124 KB. Therefore, the communication cost of our scheme is higher than that of [24], [25], and [30]. Both our scheme and [24] are unaffected by the number of challenged blocks, while the total communication costs of [25] and [30] increase linearly with the number of challenged blocks. This difference arises because the challenge size in our scheme and [24] is fixed at 640 bits, whereas in [25] and [30] it depends on the number of challenged blocks. The additional cost in our scheme is mainly due to the partial authentication tags that the AS must send to the DO. Nevertheless, this extra overhead is acceptable, as it significantly enhances security and controllability by requiring the DO’s confirmation during tag finalization. Moreover, unlike [25] and [30], whose communication costs grow linearly with the challenge size, our scheme maintains a stable cost, thus offering better scalability in large-scale auditing scenarios.

5) *Storage Cost*: The total storage cost is mainly determined by the data size stored by each entity. As shown in Fig. 7, when the cloud-stored file is divided into 1000 blocks and the number of challenged blocks ranges from 200 to 1000, the total storage costs of our scheme and [24] remain constant at 84.12 and 84.208 KB, respectively. In contrast, the total storage cost of [25] increases from 92.2–124.2 KB, and that of [30] increases from 92.048–124.048 KB. It can be observed that the total storage cost of our scheme is comparable to that of [24], and lower than those of [25] and [30]. In this experiment, the sizes of data blocks and their authentication tags are the same across all schemes. However, in [25] and [30], the storage cost of the BC/TPA is linearly related to the number of challenged blocks, and therefore the total storage cost increases as the number of challenged blocks grows from 200 to 1000.

VIII. CONCLUSION

In this article, we constructed an assistant-based integrity auditing scheme with a privacy protection function for cloud storage, particularly focusing on the computational burden of DO in generating authentication tags by introducing an Assistant. To further enhance security and privacy during the audit process, the scheme introduces a transparent audit mechanism based on Blockchain. In the meantime, the addition of timestamps and PRF increases the randomness of challenge messages, thereby audit security is enhanced. The proposed scheme achieves robust security in terms of cloud audit reliability, tag collaborative computing security, and privacy protection. Performance analysis indicates that the scheme offers advantages over similar solutions in terms of computational, storage, and communication costs.

ACKNOWLEDGMENT

The authors would like to thank the anonymous referees for their valuable suggestions and comments.

REFERENCES

- [1] X. Zheng, W. Shen, Y. Su, and Y. Gao, "DIADD: Secure deduplication and efficient data integrity auditing with data dynamics for cloud storage," *IEEE Trans. Netw. Service Manage.*, vol. 22, no. 1, pp. 299–316, Feb. 2025.
- [2] K. Zhang et al., "A reliable distributed-cloud storage based on permissioned blockchain," *IEEE Trans. Services Comput.*, vol. 18, no. 3, pp. 1216–1231, May 2025.
- [3] Q. Zhang, S. Qian, J. Cui, H. Zhong, F. Wang, and D. He, "Blockchain-based privacy-preserving deduplication and integrity auditing in cloud storage," *IEEE Trans. Comput.*, vol. 74, no. 5, pp. 1717–1729, May 2025.
- [4] K. Ren, C. Wang, and Q. Wang, "Security challenges for the public cloud," *IEEE Internet Comput.*, vol. 16, no. 1, pp. 69–73, Jan. 2012.
- [5] M. Green, "The threat in the cloud," *IEEE Secur. Privacy*, vol. 11, no. 1, pp. 86–89, Jan. 2013.
- [6] H. Shacham and B. Waters, "Compact proofs of retrievability," *J. Cryptol.*, vol. 26, no. 3, pp. 442–483, Jul. 2013.
- [7] D. He, Y. Zhang, D. Wang, and K.-K.-R. Choo, "Secure and efficient two-party signing protocol for the identity-based signature scheme in the IEEE P1363 standard for public key cryptography," *IEEE Trans. Dependable Secure Comput.*, vol. 17, no. 5, pp. 1124–1132, Sep. 2020.
- [8] M. Mambo, E. Okamoto, and K. Usuda, "Proxy signatures: Delegation of the power to sign messages," *IEICE Trans. Fundam.*, vol. 79, no. 9, pp. 1338–1354, Sep. 1996.
- [9] H. Tian et al., "Dynamic-hash-table based public auditing for secure cloud storage," *IEEE Trans. Services Comput.*, vol. 10, no. 5, pp. 701–714, Sep. 2017.
- [10] S. G. Worku, C. Xu, J. Zhao, and X. He, "Secure and efficient privacy-preserving public auditing scheme for cloud storage," *Comput. Electr. Eng.*, vol. 40, no. 5, pp. 1703–1713, Jul. 2014.
- [11] G. Ateniese et al., "Provable data possession at untrusted stores," in *Proc. 14th ACM Conf. Comput. Commun. Secur.*, Alexandria, VA, USA, Oct. 2007, pp. 598–609.
- [12] G. Ateniese, R. Di Pietro, L. V. Mancini, and G. Tsudik, "Scalable and efficient provable data possession," in *Proc. 4th Int. Conf. Secur. Privacy Commun. Netw.*, Sep. 2008, pp. 1–10.
- [13] Q. Wang, C. Wang, K. Ren, W. Lou, and J. Li, "Enabling public auditability and data dynamics for storage security in cloud computing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 22, no. 5, pp. 847–859, May 2011.
- [14] S. Nakamoto and A. Bitcoin. (2008). *A Peer-to-Peer Electronic Cash System*. [Online]. Available: <https://bitcoin.org/bitcoin.pdf>
- [15] Y. Zhang, J. Jiang, X. Dong, L. Wang, and Y. Xiang, "BeDCV: Blockchain-enabled decentralized consistency verification for cross-chain calculation," *IEEE Trans. Cloud Comput.*, vol. 11, no. 3, pp. 2273–2284, Jul. 2023.
- [16] C. Wang, Y. Sun, B. Liu, L. Xue, and X. Guan, "Blockchain-based dynamic cloud data integrity auditing via non-leaf node sampling of rank-based Merkle hash tree," *IEEE Trans. Netw. Sci. Eng.*, vol. 11, no. 5, pp. 3931–3942, Sep. 2024.
- [17] I. U. Din, A. Almogren, Z. Han, and M. Guizani, "Ensuring privacy and integrity in IoT supply chains through blockchain and homomorphic encryption," *IEEE Internet Things J.*, vol. 12, no. 13, pp. 25547–25558, Jul. 2025.
- [18] Q. Lin, J. Li, Z. Huang, W. Chen, and J. Shen, "A short linearly homomorphic proxy signature scheme," *IEEE Access*, vol. 6, pp. 12966–12972, 2018.
- [19] C. Wang and B. Wu, "A linear homomorphic proxy signature scheme based on blockchain for Internet of Things," *Comput. Model. Eng. Sci.*, vol. 136, no. 2, pp. 1857–1878, Feb. 2023.
- [20] H. Zhang, X. Zhang, Y. Zhang, X. Wang, and Q. Liu, "Blockchain-based proxy-oriented data integrity checking mechanism in cloud-assisted intelligent transportation systems," *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 11, pp. 17366–17381, Nov. 2024.
- [21] C. Wang, S. S. M. Chow, Q. Wang, K. Ren, and W. Lou, "Privacy-preserving public auditing for secure cloud storage," *IEEE Trans. Comput.*, vol. 62, no. 2, pp. 362–375, Feb. 2013.
- [22] W. Shen, J. Qin, J. Yu, R. Hao, and J. Hu, "Enabling identity-based integrity auditing and data sharing with sensitive information hiding for secure cloud storage," *IEEE Trans. Inf. Forensics Security*, vol. 14, no. 2, pp. 331–346, Feb. 2019.
- [23] X. Gao, J. Yu, Y. Chang, H. Wang, and J. Fan, "Checking only when it is necessary: Enabling integrity auditing based on the keyword with sensitive information privacy for encrypted cloud data," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 6, pp. 3774–3789, Nov. 2022.
- [24] M. Wang, J. Yu, W. Shen, and R. Hao, "Privacy-preserving time-based auditing for secure cloud storage," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 7866–7878, 2024.
- [25] X. Hu, J. Chang, T. Ahmad, F. Zhang, and Y. Zhang, "Identity-based integrity auditing scheme with sensitive information hiding for proxy-server-assisted cloud storage applications," *IEEE Internet Things J.*, vol. 12, no. 6, pp. 6673–6684, Mar. 2025.
- [26] Y. Lindell, "Fast secure two-party ECDSA signing," *J. Cryptol.*, vol. 34, no. 4, Oct. 2021, Art. no. 44.
- [27] Y. Mu, H. Xu, P. Li, and T. Ma, "Secure two-party SM9 signing," *Sci. China Inf. Sci.*, vol. 63, no. 8, Aug. 2020, Art. no. 189101.
- [28] Y. Liu, F. Qi, C. Peng, and M. Luo, "Asymmetric secure multi-party signing protocol for the identity-based signature scheme in the IEEE P1363 standard for public key cryptography," in *Proc. Int. Symp. Emerg. Inf. Secur. Appl.*, Oct. 2022, pp. 1–20.
- [29] S. G. Liu, R. Liu, and S. Y. Rao, "Secure and efficient two-party collaborative SM9 signature scheme suitable for smart home," *J. King Saud Univ.-Comput. Inf. Sci.*, vol. 34, no. 7, pp. 4022–4030, Jul. 2022.
- [30] Y. Huang, W. Shen, and J. Qin, "Certificateless cloud storage auditing supporting data ownership transfer," *Comput. Secur.*, vol. 139, Apr. 2024, Art. no. 103738.